

Quiero implementar en este repositorio un flujo de CI/CD para versionar y publicar las reglas Semgrep y YARA que vive en este repo. Son dos pipelines independientes que conviven en el mismo repositorio:

- SEMGREP: reglas en la carpeta SEMGREP/, organizadas por subcarpeta de lenguaje (SEMGREP/python, SEMGREP/javascript, SEMGREP/dockerfile, etc.).
- YARA: pipeline en cadena de tres piezas:
 1. yara_rules/Yara-Rules-Repo/ — submódulo git (apunta a Yara-Rules/rules), organizado por carpeta de categoría de amenaza (antidebug_antivm, malware, webshells, maldocs, cve_rules, exploit_kits, crypto, email, packers, capabilities, mobile_malware, utils, deprecated, etc.). Nunca se modifica directamente.
 2. yara_scores_output/yara_rule_scores.json — decide qué reglas del submódulo se publican y con qué severity_score/confidence_score/exploitability_score/risk_score (fórmula en .claude/criterioPuntuacionReglas.md).
 3. scripts/generate_scored_yara.py — lee cada regla referenciada en el JSON de scores desde el submódulo (via git show HEAD:), le inyecta la puntuación en el bloque meta, incluye las private rules necesarias como dependencia estructural, y escribe el resultado final en dist/yara_scored/.
 - Ya existe un workflow .github/workflows/publish-scored-yara.yml que regenera dist/yara_scored/ (en runner Ubuntu, no Windows) cada vez que cambia el JSON de scores, el submódulo, o el script, y commitea el resultado. NO lo dupliques ni lo reemplaces — este nuevo workflow de versionado/release debe ejecutarse DESPUÉS de que dist/yara_scored/ ya esté actualizado y commiteado (puede depender de ese workflow o dispararse también con push a main una vez el commit de dist/yara_scored/ ya está aplicado).

El objetivo de este nuevo workflow es que cada cambio validado en main (tanto en SEMGREP/ como en dist/yara_scored/) genere una versión nueva conjunta y notifique automáticamente a un repo consumidor privado, sin que ese consumidor tenga que hacer polling.

Necesito un workflow de GitHub Actions (.github/workflows/release-rules.yml) que se dispare en push a main y haga lo siguiente, en este orden:

1. VALIDACIÓN

- Ejecutar `semgrep --validate` sobre todo el contenido de SEMGREP/.
- Validar la sintaxis de las reglas YARA en dist/yara_scored/ (usando yara -w o una librería equivalente en Python). No es necesario re-validar el submódulo origen, solo el artefacto final ya publicado.
- Si algo falla, el workflow debe fallar aquí y no continuar (no crear tag ni notificar).

2. DETECCIÓN DE CAMBIOS

- Comparar el commit actual contra el commit anterior (git diff) y determinar:
 - a) qué subcarpetas de lenguaje dentro de SEMGREP/ tienen cambios (languages_changed)
 - b) qué subcarpetas de categoría dentro de dist/yara_scored/ tienen cambios (yara_categories_changed)
- Guardar ambas listas para usarlas en el payload del dispatch más adelante.

3. GENERAR manifest.json en la raíz del repo (o en un directorio /dist) con esta estructura:

```
{
  "version": "vX.Y.Z",
  "generated_at": "<timestamp ISO 8601>",
  "hashes": {
    "semgrep": {
      "python": "sha256:<hash calculado solo sobre SEMGREP/python>",
      "javascript": "sha256:<hash calculado solo sobre SEMGREP/javascript>",
      "dockerfile": "sha256:<hash calculado solo sobre SEMGREP/dockerfile>"
    },
    "yara": {
      "antidebug_antivm": "sha256:<hash calculado solo sobre dist/yara_scored/antidebug_antivm>",
      "webshells": "sha256:<hash calculado solo sobre dist/yara_scored/webshells>"
    }
  },
  // una entrada por cada subcarpeta existente (de lenguaje en SEMGREP/, o de categoría en
  // dist/yara_scored/), no solo las que cambiaron en este push
  "languages_changed": ["python"],
  "yara_categories_changed": ["webshells"],
  "rule_count": { "semgrep": 47, "yara": 694 }
}
```

IMPORTANTE:

- Usa un hash POR CLAVE (por lenguaje dentro de "semgrep", por categoría dentro de "yara"), nunca un hash global de toda la carpeta. Esto es así por dos motivos: (1) evita que una colisión o discrepancia en una clave se confunda con problemas en otra, permitiendo identificar exactamente qué conjunto de reglas falló; (2) el consumidor hace sparse-checkout/descarga parcial (solo el/los lenguaje(s) o categoría(s) que necesita), así que tiene que poder verificar la integridad de esa descarga parcial sin traerse el repo completo.
- El hash de YARA se calcula SIEMPRE sobre dist/yara_scored// (el artefacto final ya publicado con la puntuación inyectada), NUNCA sobre yara_scores_output/yara_rule_scores.json ni sobre el submódulo. Es lo que el consumidor va a descargar y usar realmente, así que es lo único cuya integridad importa verificar end-to-end.
- Para cada clave (lenguaje o categoría), usa un método determinista de hash (concatenar el contenido de todos los ficheros de esa subcarpeta, ordenados alfabéticamente por ruta, y calcular sha256 sobre esa concatenación). Documenta el método exacto en un comentario, porque el repo consumidor tiene que poder recalcularlo el mismo hash de forma independiente para cada clave.

4. VERSIONADO SEMÁNTICO (SemVer, formato vMAJOR.MINOR.PATCH)

- El bump de versión se determina mediante una ETIQUETA MANUAL en el mensaje del commit (o del PR) que dispara el push a main. Busca en el mensaje del commit/PR alguna de estas etiquetas, en este orden de prioridad:
[major] → sube MAJOR (resetea MINOR y PATCH a 0). Reservado para cambios incompatibles: eliminar una regla, cambiar el formato del manifest.json, reestructurar carpetas de lenguaje o de categoría YARA.

[minor] → sube MINOR (resetea PATCH a 0). Para reglas nuevas (Semgrep o YARA), soporte de un lenguaje nuevo, o publicación de una categoría YARA nueva en `yara_rule_scores.json`.
[patch] → sube PATCH. Para correcciones de reglas existentes (falsos positivos, ajustes de puntuación, typos), en Semgrep o en YARA.

- Aplica el mismo esquema de versión a ambos pipelines conjuntamente (una sola versión `vX.Y.Z` para todo el manifest), no versiones separadas para Semgrep y YARA.
- Si el commit/PR no incluye ninguna etiqueta, asume [patch] por defecto y deja constancia explícita en el log del workflow (por ejemplo: "No se encontró etiqueta de versión, aplicando PATCH por defecto") para que quede claro que fue una decisión automática y no una etiqueta explícita.
- No implementes inferencia automática por Conventional Commits ni por diff de contenido — la etiqueta manual es la única fuente de verdad para el bump, al menos por ahora.
- Lee el último tag existente (`vX.Y.Z`) con `git describe --tags --abbrev=0` (o equivalente) y aplica el incremento correspondiente sobre ese valor.

5. CREAR TAG Y RELEASE

- Crear el tag Git con la versión calculada.
- Crear una GitHub Release asociada, adjuntando `manifest.json` como release asset y generando release notes automáticas con los cambios (indicando qué lenguajes Semgrep y qué categorías YARA cambiaron).

6. NOTIFICAR AL REPO CONSUMIDOR (repository_dispatch)

- Hacer un POST a `https://api.github.com/repos//<REPO_CONSUMIDOR>/dispatches`
- `event_type`: "rules-updated"
- `client_payload`: { "version": "vX.Y.Z", "hashes": { "semgrep": {...}, "yara": {...} }, "languages_changed": [...], "yara_categories_changed": [...] }
(incluye en "hashes" al menos las entradas correspondientes a `languages_changed` y `yara_categories_changed`; puedes incluir el objeto completo del manifest si prefieres simplificar el payload)
- El token para esta llamada debe leerse de un secret llamado `DISPATCH_TOKEN` (fine-grained PAT, scope mínimo: solo permiso para disparar `repository_dispatch` en ese repo concreto). No lo hardcodees ni lo imprimas en logs (cuidado con `set -x`).

Requisitos adicionales:

- Si cualquier paso falla, el job debe marcar el workflow como fallido y NO debe llegar a la fase de dispatch.
- Añade comentarios explicando cada paso, ya que este repo forma parte de un proyecto académico (TFG/ asignatura de ciberseguridad) y quiero poder explicar cada decisión.
- No asumas nombres de secrets ni de repos que no te haya confirmado — pregúntame si falta algún dato (por ejemplo el owner/nombre exacto del repo consumidor).